The Second Eye and the Assembler: Stage Three of a Verified Reader, Movement Two

Working paper v0.1, Project secret3, August 11, 2026

Abstract

Movement two internalizes the S clause. The bench grows by exactly what the deeper spine demands and nothing more: beta under two and under three outer substitutions (absSubSteps2, absSubSteps3, the same induction as their ancestor with more wrapping), the packaged three- and four-argument betas (abs3Beta, abs4Beta), and one congruence lemma for reducing inside a result tag (jTagCong). On this bench stands readS, the second eye: a closed term of four nested cases that inspects any quoted program and answers whether its root is an S redex. The novelty over movement one is constructive: the S contractum does not exist inside the input, so the firing branch manufactures it, three internal constructor calls assembling the quote of (a g)(b g) from the three quoted cargos, with the argument duplicated, the share performed on descriptions. The firing theorem readSFires is a twelve-link composition ending in a congruence-wrapped chain of three qMkBeta instances reducing inside the tag; eight silence theorems cover the non-firing shapes; together the nine match the engine's root-S clause exactly, quantified over all subcodes. Every side condition of every multi-argument beta, sixteen occurrences across the movement, is discharged by quoteClosed. The dispatcher is now pure plumbing; only the knot remains.

1. What is new: the reader builds

Movement one's eye extracted: the answer it returned, the kept child, already existed inside the input quote. The S redex denies that comfort. Its contractum, (a g)(b g), shares the argument into both branches, so the eye must construct a quote that was never there, and duplication of g is not optional, it is the meaning of S. The internal constructor qMk from movement one does the manufacturing: the firing branch is jTag applied to qMk (qMk a g) (qMk b g), written with variables, spliced by the betas, and then reduced, inside the tag, by three qMkBeta instances lifted under congruences. This is the first time bench lemmas compose under context operators, and it was taken deliberately: the knot of movement four will interleave such congruence-lifted chains with unfoldings, and the pattern now has a clean, compiled precedent, the last nine links of readSFires.

2. The bench extension, mechanical by design

The deeper spine threads outer values down: the level-one continuation must know the root argument, the level-two continuation the argument and the second cargo, so the case branches are partially applied abstractions and the betas run at arity three and four. The two new substitution lemmas repeat absSubSteps with one and two more sub1 wrappers; their leaf cases stack the wrappers over the same four-way name analysis, and their side conditions grow arithmetically, one per outer name per argument: abs3Beta carries three, abs4Beta six. All sixteen condition sites in the movement are discharged by quoteClosed, which has now justified its existence twice over. Nothing else was added; the movement's whole cost is wrapping.

3. The eye and its theorems

```
readSFires : Steps (readS (quote (QApp (QApp (QApp QS a) b) g)))
                   (jTag (quote (QApp (QApp a g) (QApp b g))))
```

with readSAtomS, readSAtomK, readSL1S, readSL1K, readSL2S, readSL2K, readSL3K, readSL3Deep delivering nothing on the eight non-firing shapes, each a prefix or variant of the firing chain. Coverage is exact against the engine: the nine theorems are the root-S clause, shape for shape, quantified over every subcode, and the compiled module checks them as it checks everything else, silently and completely.

4. Standing and remaining

Standing after two movements: the alphabet, the engine as specification, the bench through arity four, tags, the constructor, and both eyes, every piece composition-verified. Movement three, the dispatcher, is the tag-case plumbing that tries the K eye and, on nothing, the S eye, agreeing with the engine's root clauses by construction from the eyes' theorems; no new machinery is expected. Movement four remains what it was named: recursion through the fixed point, correctness across its unfoldings, the summit's true face, now with every approach path paved.

Appendix A: Methods

All theorems are in the shared proof module Proofs.idr, 2,754 lines after this paper's additions, %default total, no postulates, no holes; idris2 —check Proofs.idr is the entire verification.

Appendix B: The second-eye section (module part U)

```
--------------------------------------------------------------------
-- Tier 2, part U: the second eye and the assembler (C5, movement two)
--------------------------------------------------------------------

||| beta under two outer substitutions
absSubSteps2 : (x, y, z : String) -> (b, u, v, w : Term) ->
  occ x w = 0 -> occ y w = 0 ->
  Steps (App (sub1 x u (sub1 y v (abs z b))) w)
        (sub1 x u (sub1 y v (sub1 z w b)))
absSubSteps2 x y z (Var n) u v w px py with (z == n)
  absSubSteps2 x y z (Var n) u v w px py | True =
    rewrite sub1NoOcc y v w py in
    rewrite sub1NoOcc x u w px in
    skIdentity CK w
  absSubSteps2 x y z (Var n) u v w px py | False with (y == n)
    absSubSteps2 x y z (Var n) u v w px py | False | True =
      kTrue (sub1 x u v) w
    absSubSteps2 x y z (Var n) u v w px py | False | False with (x == n)
      absSubSteps2 x y z (Var n) u v w px py | False | False | True =
        kTrue u w
      absSubSteps2 x y z (Var n) u v w px py | False | False | False =
        kTrue (Var n) w
absSubSteps2 x y z CK u v w px py = kTrue CK w
absSubSteps2 x y z CS u v w px py = kTrue CS w
absSubSteps2 x y z (App a b) u v w px py =
  Then RedS $
  stepsTrans (appLs (absSubSteps2 x y z a u v w px py))
             (stepsAppR (absSubSteps2 x y z b u v w px py))
```

```
||| beta under three outer substitutions
absSubSteps3 : (x, y, z, n0 : String) -> (b, u, v, w, t : Term) ->
  occ x t = 0 -> occ y t = 0 -> occ z t = 0 ->
  Steps (App (sub1 x u (sub1 y v (sub1 z w (abs n0 b)))) t)
        (sub1 x u (sub1 y v (sub1 z w (sub1 n0 t b))))
absSubSteps3 x y z n0 (Var n) u v w t px py pz with (n0 == n)
  absSubSteps3 x y z n0 (Var n) u v w t px py pz | True =
    rewrite sub1NoOcc z w t pz in
    rewrite sub1NoOcc y v t py in
    rewrite sub1NoOcc x u t px in
    skIdentity CK t
  absSubSteps3 x y z n0 (Var n) u v w t px py pz | False with (z == n)
    absSubSteps3 x y z n0 (Var n) u v w t px py pz | False | True =
      kTrue (sub1 x u (sub1 y v w)) t
    absSubSteps3 x y z n0 (Var n) u v w t px py pz | False | False
      with (y == n)
      absSubSteps3 x y z n0 (Var n) u v w t px py pz | False | False | True =
        kTrue (sub1 x u v) t
      absSubSteps3 x y z n0 (Var n) u v w t px py pz
        | False | False | False with (x == n)
        absSubSteps3 x y z n0 (Var n) u v w t px py pz
          | False | False | False | True = kTrue u t
        absSubSteps3 x y z n0 (Var n) u v w t px py pz
          | False | False | False | False = kTrue (Var n) t
absSubSteps3 x y z n0 CK u v w t px py pz = kTrue CK t
absSubSteps3 x y z n0 CS u v w t px py pz = kTrue CS t
absSubSteps3 x y z n0 (App a b) u v w t px py pz =
  Then RedS $
  stepsTrans (appLs (absSubSteps3 x y z n0 a u v w t px py pz))
             (stepsAppR (absSubSteps3 x y z n0 b u v w t px py pz))

||| three- and four-argument beta for the vanishing engine
abs3Beta : (x, y, z : String) -> (b, u, v, w : Term) ->
  occ x v = 0 -> occ x w = 0 -> occ y w = 0 ->
  Steps (App (App (App (abs x (abs y (abs z b))) u) v) w)
        (sub1 x u (sub1 y v (sub1 z w b)))
abs3Beta x y z b u v w pxv pxw pyw =
  stepsTrans (appLs (abs2Beta x y (abs z b) u v pxv))
             (absSubSteps2 x y z b u v w pxw pyw)

abs4Beta : (x, y, z, n0 : String) -> (b, u, v, w, t : Term) ->
  occ x v = 0 -> occ x w = 0 -> occ y w = 0 ->
  occ x t = 0 -> occ y t = 0 -> occ z t = 0 ->
  Steps (App (App (App (App (abs x (abs y (abs z (abs n0 b)))) u) v) w) t)
        (sub1 x u (sub1 y v (sub1 z w (sub1 n0 t b))))
abs4Beta x y z n0 b u v w t pxv pxw pyw pxt pyt pzt =
  stepsTrans (appLs (abs3Beta x y z (abs n0 b) u v w pxv pxw pyw))
             (absSubSteps3 x y z n0 b u v w t pxt pyt pzt)

||| congruence into the payload of a just tag
jTagCong : {x, y : Term} -> Steps x y ->
  Steps (Proofs.jTag x) (Proofs.jTag y)
jTagCong ss =
  stepsAppR {l = CK}
```

```
    (stepsAppR {l = App CS Proofs.iC}
      (stepsAppR {l = CK} ss))

||| the assembled S contractum, reduced inside the tag
fireT : Term
fireT = Proofs.jTag
  (App (App Proofs.qMk (App (App Proofs.qMk (Var "aa")) (Var "gg")))
       (App (App Proofs.qMk (Var "bb")) (Var "gg")))

h4Term : Term
h4Term = abs "gg" (abs "bb" (abs "w" (abs "aa"
  (App (App (App (Var "w") Proofs.fireT) Proofs.nTag) Proofs.drop2))))

g3Term : Term
g3Term = abs "g" (abs "p" (abs "b"
  (App (App (App (Var "p") Proofs.nTag) Proofs.nTag)
       (App (App Proofs.h4Term (Var "g")) (Var "b")))))

fTermS : Term
fTermS = abs "l" (abs "r"
  (App (App (App (Var "l") Proofs.nTag) Proofs.nTag)
       (App Proofs.g3Term (Var "r"))))

||| the second eye: does the root of this quoted program fire as S?
readS : Term
readS = abs "q0"
  (App (App (App (Var "q0") Proofs.nTag) Proofs.nTag) Proofs.fTermS)

||| the eye fires and assembles: a quoted S redex yields the quoted
||| contractum, built by the internal constructor, share performed
||| on descriptions
readSFires : (a, b, g : Code) ->
  Steps (App Proofs.readS
          (Proofs.quote (QApp (QApp (QApp QS a) b) g)))
        (Proofs.jTag (Proofs.quote (QApp (QApp a g) (QApp b g))))
readSFires a b g =
  stepsTrans (absCorrect "q0"
    (App (App (App (Var "q0") Proofs.nTag) Proofs.nTag) Proofs.fTermS)
    (quote (QApp (QApp (QApp QS a) b) g))) $
  stepsTrans (qAppCase (QApp (QApp QS a) b) g nTag nTag fTermS) $
  stepsTrans (abs2Beta "l" "r"
    (App (App (App (Var "l") Proofs.nTag) Proofs.nTag)
         (App Proofs.g3Term (Var "r")))
    (quote (QApp (QApp QS a) b)) (quote g) (quoteClosed "l" g)) $
  stepsTrans (qAppCase (QApp QS a) b nTag nTag
    (App g3Term (quote g))) $
  stepsTrans (abs3Beta "g" "p" "b"
    (App (App (App (Var "p") Proofs.nTag) Proofs.nTag)
         (App (App Proofs.h4Term (Var "g")) (Var "b")))
    (quote g) (quote (QApp QS a)) (quote b)
    (quoteClosed "g" (QApp QS a)) (quoteClosed "g" b)
    (quoteClosed "p" b)) $
  stepsTrans (qAppCase QS a nTag nTag
    (App (App h4Term (quote g)) (quote b))) $
  stepsTrans (abs4Beta "gg" "bb" "w" "aa"
```

```
    (App (App (App (Var "w") Proofs.fireT) Proofs.nTag) Proofs.drop2)
    (quote g) (quote b) (quote QS) (quote a)
    (quoteClosed "gg" b) (quoteClosed "gg" QS) (quoteClosed "bb" QS)
    (quoteClosed "gg" a) (quoteClosed "bb" a) (quoteClosed "w" a)) $
  stepsTrans (qSCase
    (jTag (App (App qMk (App (App qMk (quote a)) (quote g)))
               (App (App qMk (quote b)) (quote g))))
    nTag drop2) $
  jTagCong $
  stepsTrans (appLs (stepsAppR {l = qMk} (qMkBeta a g))) $
  stepsTrans (stepsAppR {l = App qMk (quote (QApp a g))}
               (stepsAppR {l = qMk} (qMkBeta b g))) $
  qMkBeta (QApp a g) (QApp b g)

||| and stays silent on every non-firing shape
readSAtomS : Steps (App Proofs.readS (Proofs.quote QS)) Proofs.nTag
readSAtomS =
  stepsTrans (absCorrect "q0"
    (App (App (App (Var "q0") Proofs.nTag) Proofs.nTag) Proofs.fTermS)
    (quote QS)) $
  qSCase nTag nTag fTermS

readSAtomK : Steps (App Proofs.readS (Proofs.quote QK)) Proofs.nTag
readSAtomK =
  stepsTrans (absCorrect "q0"
    (App (App (App (Var "q0") Proofs.nTag) Proofs.nTag) Proofs.fTermS)
    (quote QK)) $
  qKCase nTag nTag fTermS

readSL1S : (g : Code) ->
  Steps (App Proofs.readS (Proofs.quote (QApp QS g))) Proofs.nTag
readSL1S g =
  stepsTrans (absCorrect "q0"
    (App (App (App (Var "q0") Proofs.nTag) Proofs.nTag) Proofs.fTermS)
    (quote (QApp QS g))) $
  stepsTrans (qAppCase QS g nTag nTag fTermS) $
  stepsTrans (abs2Beta "l" "r"
    (App (App (App (Var "l") Proofs.nTag) Proofs.nTag)
         (App Proofs.g3Term (Var "r")))
    (quote QS) (quote g) (quoteClosed "l" g)) $
  qSCase nTag nTag (App g3Term (quote g))

readSL1K : (g : Code) ->
  Steps (App Proofs.readS (Proofs.quote (QApp QK g))) Proofs.nTag
readSL1K g =
  stepsTrans (absCorrect "q0"
    (App (App (App (Var "q0") Proofs.nTag) Proofs.nTag) Proofs.fTermS)
    (quote (QApp QK g))) $
  stepsTrans (qAppCase QK g nTag nTag fTermS) $
  stepsTrans (abs2Beta "l" "r"
    (App (App (App (Var "l") Proofs.nTag) Proofs.nTag)
         (App Proofs.g3Term (Var "r")))
    (quote QK) (quote g) (quoteClosed "l" g)) $
  qKCase nTag nTag (App g3Term (quote g))
```

```
readSL2S : (a, g : Code) ->
  Steps (App Proofs.readS (Proofs.quote (QApp (QApp QS a) g))) Proofs.nTag
readSL2S a g =
  stepsTrans (absCorrect "q0"
    (App (App (App (Var "q0") Proofs.nTag) Proofs.nTag) Proofs.fTermS)
    (quote (QApp (QApp QS a) g))) $
  stepsTrans (qAppCase (QApp QS a) g nTag nTag fTermS) $
  stepsTrans (abs2Beta "l" "r"
    (App (App (App (Var "l") Proofs.nTag) Proofs.nTag)
         (App Proofs.g3Term (Var "r")))
    (quote (QApp QS a)) (quote g) (quoteClosed "l" g)) $
  stepsTrans (qAppCase QS a nTag nTag (App g3Term (quote g))) $
  stepsTrans (abs3Beta "g" "p" "b"
    (App (App (App (Var "p") Proofs.nTag) Proofs.nTag)
         (App (App Proofs.h4Term (Var "g")) (Var "b")))
    (quote g) (quote QS) (quote a)
    (quoteClosed "g" QS) (quoteClosed "g" a) (quoteClosed "p" a)) $
  qSCase nTag nTag (App (App h4Term (quote g)) (quote a))

readSL2K : (a, g : Code) ->
  Steps (App Proofs.readS (Proofs.quote (QApp (QApp QK a) g))) Proofs.nTag
readSL2K a g =
  stepsTrans (absCorrect "q0"
    (App (App (App (Var "q0") Proofs.nTag) Proofs.nTag) Proofs.fTermS)
    (quote (QApp (QApp QK a) g))) $
  stepsTrans (qAppCase (QApp QK a) g nTag nTag fTermS) $
  stepsTrans (abs2Beta "l" "r"
    (App (App (App (Var "l") Proofs.nTag) Proofs.nTag)
         (App Proofs.g3Term (Var "r")))
    (quote (QApp QK a)) (quote g) (quoteClosed "l" g)) $
  stepsTrans (qAppCase QK a nTag nTag (App g3Term (quote g))) $
  stepsTrans (abs3Beta "g" "p" "b"
    (App (App (App (Var "p") Proofs.nTag) Proofs.nTag)
         (App (App Proofs.h4Term (Var "g")) (Var "b")))
    (quote g) (quote QK) (quote a)
    (quoteClosed "g" QK) (quoteClosed "g" a) (quoteClosed "p" a)) $
  qKCase nTag nTag (App (App h4Term (quote g)) (quote a))

readSL3K : (a, b, g : Code) ->
  Steps (App Proofs.readS
          (Proofs.quote (QApp (QApp (QApp QK a) b) g)))
        Proofs.nTag
readSL3K a b g =
  stepsTrans (absCorrect "q0"
    (App (App (App (Var "q0") Proofs.nTag) Proofs.nTag) Proofs.fTermS)
    (quote (QApp (QApp (QApp QK a) b) g))) $
  stepsTrans (qAppCase (QApp (QApp QK a) b) g nTag nTag fTermS) $
  stepsTrans (abs2Beta "l" "r"
    (App (App (App (Var "l") Proofs.nTag) Proofs.nTag)
         (App Proofs.g3Term (Var "r")))
    (quote (QApp (QApp QK a) b)) (quote g) (quoteClosed "l" g)) $
  stepsTrans (qAppCase (QApp QK a) b nTag nTag
    (App g3Term (quote g))) $
  stepsTrans (abs3Beta "g" "p" "b"
    (App (App (App (Var "p") Proofs.nTag) Proofs.nTag)
```

```
          (App (App Proofs.h4Term (Var "g")) (Var "b")))
    (quote g) (quote (QApp QK a)) (quote b)
    (quoteClosed "g" (QApp QK a)) (quoteClosed "g" b)
    (quoteClosed "p" b)) $
  stepsTrans (qAppCase QK a nTag nTag
    (App (App h4Term (quote g)) (quote b))) $
  stepsTrans (abs4Beta "gg" "bb" "w" "aa"
    (App (App (App (Var "w") Proofs.fireT) Proofs.nTag) Proofs.drop2)
    (quote g) (quote b) (quote QK) (quote a)
    (quoteClosed "gg" b) (quoteClosed "gg" QK) (quoteClosed "bb" QK)
    (quoteClosed "gg" a) (quoteClosed "bb" a) (quoteClosed "w" a)) $
  qKCase
    (jTag (App (App qMk (App (App qMk (quote a)) (quote g)))
               (App (App qMk (quote b)) (quote g))))
    nTag drop2

readSL3Deep : (p, q, a, b, g : Code) ->
  Steps (App Proofs.readS
          (Proofs.quote (QApp (QApp (QApp (QApp p q) a) b) g)))
        Proofs.nTag
readSL3Deep p q a b g =
  stepsTrans (absCorrect "q0"
    (App (App (App (Var "q0") Proofs.nTag) Proofs.nTag) Proofs.fTermS)
    (quote (QApp (QApp (QApp (QApp p q) a) b) g))) $
  stepsTrans (qAppCase (QApp (QApp (QApp p q) a) b) g nTag nTag fTermS) $
  stepsTrans (abs2Beta "l" "r"
    (App (App (App (Var "l") Proofs.nTag) Proofs.nTag)
         (App Proofs.g3Term (Var "r")))
    (quote (QApp (QApp (QApp p q) a) b)) (quote g)
    (quoteClosed "l" g)) $
  stepsTrans (qAppCase (QApp (QApp p q) a) b nTag nTag
    (App g3Term (quote g))) $
  stepsTrans (abs3Beta "g" "p" "b"
    (App (App (App (Var "p") Proofs.nTag) Proofs.nTag)
         (App (App Proofs.h4Term (Var "g")) (Var "b")))
    (quote g) (quote (QApp (QApp p q) a)) (quote b)
    (quoteClosed "g" (QApp (QApp p q) a)) (quoteClosed "g" b)
    (quoteClosed "p" b)) $
  stepsTrans (qAppCase (QApp p q) a nTag nTag
    (App (App h4Term (quote g)) (quote b))) $
  stepsTrans (abs4Beta "gg" "bb" "w" "aa"
    (App (App (App (Var "w") Proofs.fireT) Proofs.nTag) Proofs.drop2)
    (quote g) (quote b) (quote (QApp p q)) (quote a)
    (quoteClosed "gg" b) (quoteClosed "gg" (QApp p q))
    (quoteClosed "bb" (QApp p q))
    (quoteClosed "gg" a) (quoteClosed "bb" a) (quoteClosed "w" a)) $
  stepsTrans (qAppCase p q
    (jTag (App (App qMk (App (App qMk (quote a)) (quote g)))
               (App (App qMk (quote b)) (quote g))))
    nTag drop2) $
  drop2Case (quote p) (quote q)
```

References

1. Project secret3, ‘The First Eye’, working paper v0.1, August 11, 2026, whose bench and pattern this movement extends.

2. Project secret3, ‘A Quoted Alphabet’ and ‘The Reader’s Engine’, working papers v0.1, August 11, 2026.
3. Project secret3, research agenda v0.11, line C5.